

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a8ac22749295443c6.jsonl`  
- SHA-256: `23a9153c892e19e581ee281abc53703d16bf1abba5ce90a06710aa8151567208`  
- Source modified: `2026-06-13T19:16:58+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:15:36.707Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"shuttle_player_colocation 'pad_frac' adjacency branch and a non-None court_polygon in two_sided_motion are never exercised","severity":"med","file":"tests/test_fusion_features.py:94-106,81-89","detail":"shuttle_player_colocation's whole point is the pad_frac=0.03 'in-hand/adjacent' tolerance (_shuttle_in_box expands the box by pad_frac*frame dims). Every test point is either dead-centre inside the box (50,50) or far outside (95,95); the padding boundary (just-outside-box-but-within-pad => colocated) is never tested. I confirmed x=62 is colocated and x=64 is not for box x2=60,W=100 ? exactly the discriminating behavior that should be pinned. Separately, two_sided_motion's court_polygon argument is only ever passed as None in tests; the in-polygon filtering inside two_sided_motion (off-court persons must not establish a 'side') is untested ? I confirmed a NEAR_ONLY polygon correctly drops two_sided from 1.0 to 0.0, but nothing asserts it.","fix":"Add a colocation case with the shuttle just inside the pad halo (and one just beyond) to pin the tolerance, and add a non-None pad_frac=0 case. Add a two_sided_motion case with a polygon that excludes the far-side player and assert it returns 0.0 (off-court persons don't count as a side)."}
}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:40.205Z)

I'll verify this finding by reading the actual test file and the feature implementation. Let me start by examining the relevant files.

3. user (2026-06-13T19:15:41.166Z)

```
1      """Pure-logic unit tests for the person-cue fusion features  
(backend.pipeline.detectors.fusion_features).  
2  
3      No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path  
and
```

```

4      edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5      ""
6      from types import SimpleNamespace
7
8      import pytest
9
10     from backend.pipeline.detectors import fusion_features as ff
11
12
13     def _shuttle(frame, x, y, visible=True):
14         return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17     # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18     FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19     LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22     # ----- point_in_polygon
23
24     def test_point_in_polygon_inside_outside():
25         assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26         assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27         assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28         assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31     def test_point_in_polygon_degenerate():
32         assert ff.point_in_polygon((0.5, 0.5), None) is False
33         assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36     # ----- person_in_court
37
38     def test_person_in_court_uses_foot_point_and_polygon():
39         # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40         in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41         out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42         assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43         assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44         # None polygon => everyone counts
45         assert ff.person_in_court(out_box, 100, 100, None) is True
46         # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47         assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50     # ----- counts / ratios
51
52     def test_in_court_counts_and_players_and_ratio():
53         near = (10, 10, 30, 30) # foot (0.2,0.3) in LEFT_HALF
54         far = (60, 10, 80, 30) # foot (0.7,0.3) NOT in LEFT_HALF
55         per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56
57         # With LEFT_HALF mask: counts = [1, 1, 0, 2]
58         counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
59         assert counts == [1, 1, 0, 2]
60         assert ff.players_in_court(counts) == 1.0 # median of [0,1,1,2]
61         assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
62         assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
63         # No mask: far counts too -> [2,1,0,2]
64         assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
65         assert ff.players_in_court([]) == 0.0

```

```

65         assert ff.in_court_ratio([]) == 0.0
66
67
68     # ----- mean_player_motion
69
70     def test_mean_player_motion():
71         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
72         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74         # single frame -> 0; no shared track id -> 0
75         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79     # ----- two_sided_motion
80
81     def test_two_sided_motion():
82         net = ("y", 0.5)
83         near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
84         far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
85         both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
86         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87         one_sided = {0: [(1, near)], 1: [(2, near)]}
88         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0      # no net -> 0
90
91
92     # ----- shuttle_player_colocation
93
94     def test_shuttle_player_colocation():
95         box = (40, 40, 60, 60)
96         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99         # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
100         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101         # all-held -> 1.0
102         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109     # ----- aggregator
110
111     def test_compute_person_features_keys():
112         near = (10, 10, 30, 40)
113         far = (10, 60, 30, 90)
114         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115         shuttle = [_shuttle(0, 20, 25, True)]
116         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
court_polygon=None, net=("y", 0.5))
117
118         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
119         assert all(isinstance(v, float) for v in feats.values())
120         assert feats["two_sided_motion"] == pytest.approx(1.0)      # both sides occupied every
frame
121
122

```

```

123     def test_compute_person_features_empty():
124         feats = ff.compute_person_features({}, [], 100, 100)
125         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
126         assert all(v == 0.0 for v in feats.values())
127

```

4. user (2026-06-13T19:15:41.376Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12    - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14    - A shuttle point has `(frame, visible, x, y)` (pixels) ? the WASB
trajectory shape.
15    - `court_polygon` is normalised 0-1 `(x, y, ...)` or `None` (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?
43            if (yi > y) != (yj > y):
44                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                if x < x_cross:
46                    inside = not inside
47            j = i
48        return inside
49

```

```

50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None."""
71     if court_polygon is None:
72         return True
73     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                     frame_height: float,
78                     court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79     """Per-frame count of in-court persons (one entry per sampled frame)."""
80     return [
81         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82         for _frame_idx, dets in sorted(per_frame.items())
83     ]
84
85
86 def _median(vals: Sequence[float]) -> float:
87     if not vals:
88         return 0.0
89     s = sorted(vals)
90     n = len(s)
91     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94 def players_in_court(counts: Sequence[int]) -> float:
95     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96     return _median(list(counts))
97
98
99 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100     """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101     if not counts:
102         return 0.0
103     return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                       frame_width: float, frame_height: float) -> float:
108     """Mean per-track centre displacement between consecutive sampled frames, normalised
109     by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""

```

```

110     if frame_width <= 0 or len(per_frame) < 2:
111         return 0.0
112     frames = sorted(per_frame.keys())
113     steps: List[float] = []
114     for a, b in zip(frames, frames[1:]):
115         prev = {tid: box for tid, box in per_frame[a]}
116         for tid, box in per_frame[b]:
117             if tid in prev:
118                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121     if not steps:
122         return 0.0
123     return sum(steps) / len(steps)
124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                     frame_height: float, net: Optional[Net],
128                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134     """
135     if net is None or not per_frame:
136         return 0.0
137     axis, pos = net
138     both = 0
139     for _frame_idx, dets in per_frame.items():
140         near, far = False, False
141         for _tid, box in dets:
142             if not person_in_court(box, frame_width, frame_height, court_polygon):
143                 continue
144             fx, fy = _foot_norm(box, frame_width, frame_height)
145             coord = fx if axis == "x" else fy
146             if coord < pos:
147                 near = True
148             else:
149                 far = True
150         if near and far:
151             both += 1
152     return both / len(per_frame)
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                    pad_frac: float) -> bool:
156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?"""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                             frame_width: float, frame_height: float,
166                             pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
    """

```

```

171     if not per_frame or frame_width <= 0 or frame_height <= 0:
172         return 0.0
173     aligned = 0
174     colocated = 0
175     for p in shuttle_points or []:
176         if not getattr(p, "visible", False):
177             continue
178         dets = per_frame.get(int(p.frame))
179         if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180             continue
181         aligned += 1
182         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183             for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
197 shuttle_points,
198                                frame_width: float, frame_height: float,
199                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
200 None,
201                                net: Optional[Net] = None,
202                                min_players: int = 2) -> Dict[str, float]:
203         """Aggregate the per-window person features from per-frame detections + the in-
204 window
205 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
206         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
207         return {
208             "players_in_court": players_in_court(counts),
209             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
210 court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
217

```

5. assistant (2026-06-13T19:15:50.431Z)

Both files are loaded. Now let me carefully verify the two claims in the finding by analyzing the test coverage against the implementation. Let me also check the git diff to confirm these are the new files and look at the exact arithmetic.

The finding makes two claims. Let me verify the math precisely for each.

****Claim 1: `shuttle\_player\_colocation` pad_frac boundary never tested.****

Looking at the test (lines 94-106), the box is `(40, 40, 60, 60)` and frame is `100x100`. The `pad\_frac` default is `0.03`. So `px = py = 0.03 \* 100 = 3`. The padded box is `[37, 63] x [37, 63]`.

Test shuttle points:

- `(50, 50)` ? dead-center inside `[40,60]` ? colocated (well inside).
- `(95, 95)` ? far outside even padded `[37,63]` ? not colocated.

Let me verify the finding's specific claim that $x=62$ is colocated and $x=64$ is not. The finding says "box $x_2=60$, $W=100$ ". Padded $x_2 = 60 + 3 = 63$. So $x=62 \geq 63$? inside; $x=64 > 63$? outside. That confirms the discriminating boundary is at 63. The tests never place a shuttle in the $[60, 63]$ halo region (just-outside-box-but-within-pad). Both test points are well clear of the boundary.

Let me verify the math for the `pad\_frac` default and check the call site in `compute\_person\_features`.

6. user (2026-06-13T19:15:52.825Z)

```
M backend/config/models.py
M backend/pipeline/detectors/person_detector.py
M backend/pipeline/segmenters/__init__.py
M backend/pipeline/segmenters/trajectory_hybrid.py
M docs/README.md
M tests/test_config.py
M tests/test_trajectory_hybrid_equivalence.py
?? artifacts/
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
?? docs/QUALITY_ITERATIONS.md
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
?? scratch/PR1_BODY.md
?? scratch/PR1_HANDOFF.md
?? scratch/PR2_HANDOFF.md
?? scratch/PR2_deferred_hardening_status.patch
?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
---UNTRACKED CHECK---
error: pathspec 'backend/pipeline/detectors/fusion_features.py' did not match any file(s) known
to git
error: pathspec 'tests/test_fusion_features.py' did not match any file(s) known to git
```

7. assistant (2026-06-13T19:15:55.881Z)

Confirmed: `fusion\_features.py` and `test\_fusion\_features.py` are new untracked files. Now let me verify both claims by actually running the discriminating cases the finding proposes, to confirm the behavior is real and not already covered indirectly.

8. user (2026-06-13T19:16:04.438Z)

```
--- colocation halo boundary (pad_frac=0.03, padded box right edge=63) ---
x=62 (in halo, outside raw box): 1.0
x=64 (beyond halo): 0.0
x=62 with pad_frac=0 (raw box, x2=60): 0.0

--- two_sided_motion court_polygon filtering ---
polygon=None (far counts): 1.0
polygon=NEAR_ONLY (far dropped): 0.0
```

9. assistant (2026-06-13T19:16:10.984Z)

Both claims are empirically confirmed. The runtime behavior matches exactly what the finding describes:

1. **pad_frac halo**: x=62 (in the `(60, 63]` halo, outside the raw box) ? colocated=1.0; x=64 (beyond halo) ? 0.0; and with `pad\_frac=0`, x=62 ? 0.0. The padding boundary is genuinely a discriminating behavior, and no test exercises it.

2. **court_polygon filtering in `two\_sided\_motion`**: with `NEAR\_ONLY` polygon, the far-side player is dropped, collapsing `two\_sided` from 1.0 ? 0.0. No test passes a non-None polygon to `two\_sided\_motion`.

Now let me verify the finding's claim about test coverage gaps is accurate ? i.e., that these paths are truly never exercised across the *whole* test suite, not just the two cited test functions. Let me check whether other tests indirectly cover the halo or the polygon-filtering path in `two\_sided\_motion`.

10. user (2026-06-13T19:16:11.827Z)

```
docs\EXPERIMENT_HARNESS.md:174:2. Persist per-cue features into candidate `stats` (person:
`players_in_court`, `two_sided_motion`,
docs\EXPERIMENT_HARNESS.md:175:  `in_court_ratio`, `shuttle_player_colocation`; shuttle-state:
`net_crossings`, `direction_reversals`;
docs\MULTI_SIGNAL_FUSION_PLAN.md:136:| `two_sided_motion` | are moving players on both net-
halves? | rally = two-sided; one person feeding = one-sided |
docs\MULTI_SIGNAL_FUSION_PLAN.md:139:| `shuttle_player_colocation` | fraction of frames the
shuttle sits inside/adjacent to a person box (hand region) | held = high; in-flight = mostly
free space ? the direct held-shuttle discriminator |
docs\MULTI_SIGNAL_FUSION_PLAN.md:176:| `shuttle_player_colocation` | low (free space) | high
(near a hand) | NEW (needs the person cue) |
docs\MULTI_SIGNAL_FUSION_PLAN.md:193: this mask and the net split that `two_sided_motion` /
`net_crossings` need.
docs\MULTI_SIGNAL_FUSION_PLAN.md:274:- P3 ? feature-level (learned) fusion: add the person
features (incl. `shuttle_player_colocation`)
docs\NEXT_STEPS.md:68:  `shuttle_player_colocation` to `distill_local`/`classifier`, LOVO
lift on golden), P4 (audio cue via
docs\QUALITY_ITERATIONS.md:65:player-count-only; supply ? masked). Person features
(`players_in_court`, `two_sided_motion`,
docs\QUALITY_ITERATIONS.md:66:mean_player_motion, in_court_ratio, shuttle_player_colocation`)
persisted into candidate `stats` for P3's
backend\pipeline\detectors\fusion_features.py:126:def two_sided_motion(per_frame: Dict[int,
List[Tuple[int, BBox]]], frame_width: float,
backend\pipeline\detectors\fusion_features.py:154:def _shuttle_in_box(sx: float, sy: float, box:
BBox, frame_width: float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:155:                                pad_frac: float) -> bool:
backend\pipeline\detectors\fusion_features.py:156:    """Is the (pixel) shuttle point inside a
person box, expanded by `pad_frac` of frame
backend\pipeline\detectors\fusion_features.py:159:    px = pad_frac * frame_width
backend\pipeline\detectors\fusion_features.py:160:    py = pad_frac * frame_height
backend\pipeline\detectors\fusion_features.py:164:def shuttle_player_colocation(per_frame:
Dict[int, List[Tuple[int, BBox]]], shuttle_points,
backend\pipeline\detectors\fusion_features.py:166:                                pad_frac: float
= 0.03) -> float:
backend\pipeline\detectors\fusion_features.py:182:    if any(_shuttle_in_box(p.x, p.y, box,
frame_width, frame_height, pad_frac)
backend\pipeline\detectors\fusion_features.py:191:    "players_in_court", "two_sided_motion",
"mean_player_motion",
backend\pipeline\detectors\fusion_features.py:192:    "in_court_ratio",
"shuttle_player_colocation",
backend\pipeline\detectors\fusion_features.py:206:    "two_sided_motion":
two_sided_motion(per_frame, frame_width, frame_height, net, court_polygon),
backend\pipeline\detectors\fusion_features.py:209:    "shuttle_player_colocation":
shuttle_player_colocation(
backend\pipeline\detectors\person_detector.py:266:
`fusion_features.shuttle_player_colocation` / ``two_sided_motion`` rely on.
tests\test_fusion_features.py:79:#
```

```

----- two_sided_motion
tests\test_fusion_features.py:81: def test_two_sided_motion():
tests\test_fusion_features.py:86:     assert ff.two_sided_motion(both, 100, 100, net, None) ==
pytest.approx(0.5)
tests\test_fusion_features.py:88:     assert ff.two_sided_motion(one_sided, 100, 100, net, None)
== 0.0
tests\test_fusion_features.py:89:     assert ff.two_sided_motion(both, 100, 100, None, None) ==
0.0 # no net -> 0
tests\test_fusion_features.py:92: # -----
shuttle_player_colocation
tests\test_fusion_features.py:94: def test_shuttle_player_colocation():
tests\test_fusion_features.py:100:     assert ff.shuttle_player_colocation(per_frame, shuttle,
100, 100) == pytest.approx(0.5)
tests\test_fusion_features.py:103:     assert ff.shuttle_player_colocation(per_frame, held, 100,
100) == pytest.approx(1.0)
tests\test_fusion_features.py:105:     assert ff.shuttle_player_colocation(per_frame, [], 100,
100) == 0.0
tests\test_fusion_features.py:106:     assert ff.shuttle_player_colocation(per_frame,
[_shuttle(99, 50, 50, True)], 100, 100) == 0.0
tests\test_fusion_features.py:120:     assert feats["two_sided_motion"] == pytest.approx(1.0) #
both sides occupied every frame

```

11. assistant (2026-06-13T19:16:32.807Z)

This confirms the coverage gaps comprehensively. Let me verify the two specific call sites the finding implies are exhaustive:

```

**`two_sided_motion` calls across the entire test suite** (every match):
- Line 86: `..., net, None)` ? polygon=None
- Line 88: `..., net, None)` ? polygon=None
- Line 89: `..., None, None)` ? polygon=None
- Line 120 (`compute_person_features`): called with `court_polygon=None` (line 117)

```

So every single call to `two\_sided\_motion` in tests passes `court\_polygon=None`. The in-polygon filtering branch at `fusion\_features.py:141` (`if not person\_in\_court(...): continue`) is **never reached** with a real polygon. Confirmed.

```

**`shuttle_player_colocation` calls** (every match): lines 100, 103, 105, 106, and 209 (via
aggregator). All use the default `pad_frac=0.03`, and the shuttle points are always either dead-
center `(50,50)` inside the raw box `[40,60]`, far-out `(95,95)`, empty, or unaligned. The halo
region `(60, 63)` is never hit, and `pad_frac=0` is never passed. Confirmed.

```

Now let me weigh the severity. This is a test-coverage gap finding, not a code bug. Let me assess whether the gap is meaningful or a nitpick.

The finding correctly identifies that the **single most load-bearing behavior** of each function is the one untested:

- For `shuttle\_player\_colocation`, the docstring explicitly states the `pad\_frac` is "the 'adjacent / in-hand' tolerance" ? that halo IS the function's reason for existence (distinguishing held-but-not-perfectly-overlapping shuttle from in-flight). Testing only dead-center-in and far-out would pass even if `pad\_frac` were `0` or `0.5` or removed entirely ? the test cannot detect a regression in the tolerance.
- For `two\_sided\_motion`, the docstring/design (MULTI_SIGNAL_FUSION_PLAN line 193) notes the court mask filters off-court persons. The `person\_in\_court` guard at line 141 is dead in the test suite; a regression that dropped the in-court filter would not be caught by any `two\_sided\_motion` test.

This is a genuine, specific, actionable gap ? not a "could be cleaner" nitpick. It is also not a code-correctness bug (the implementation is correct, as my runs show). So severity is appropriately **medium** (test-quality gap on load-bearing behavior), leaning toward the lower end of med since it's test coverage rather than a shipped defect.

Let me do one final check ? confirm `person\_in\_court` itself (the polygon path) IS tested elsewhere, so the finding isn't overstating that polygon logic is entirely untested (it's specifically the **integration through `two\_sided\_motion`** that's untested).

The test file does cover `person\_in\_court` with `LEFT\_HALF` (lines 38-47) and `in\_court\_counts` with `LEFT\_HALF` (line 57). So the polygon math itself is tested in isolation. The finding is precise about this: it scopes the gap to "the in-polygon filtering *inside two_sided_motion*" ? i.e., the wiring within that specific function, not the polygon primitive. That precision is accurate and the finding does not overstate.

I have enough to render a verdict.

12. user (2026-06-13T19:16:49.257Z)

Structured output provided successfully

13. assistant (2026-06-13T19:16:57.907Z)

I've completed the adversarial verification of this fusion-P2 finding.

****Verdict: real=true, severity=low****

Both claimed coverage gaps are genuine and accurately characterized. I confirmed them by running the actual discriminating cases against the real code:

1. ****`shuttle\_player\_colocation` pad_frac halo****
(`backend/pipeline/detectors/fusion\_features.py:154-185`): With box `(40,40,60,60)` and frame `100x100`, the padded right edge is `63`. Every test shuttle point is dead-centre inside the raw `[40,60]` box, far outside at `(95,95)`, empty, or unaligned. The finding's exact example holds ? $x=62$? colocated `1.0`, $x=64$? `0.0`, and $x=62$ with `pad\_frac=0` ? `0.0`. The "in-hand/adjacent" tolerance that is the function's whole purpose is never pinned.

2. ****`two\_sided\_motion` court_polygon filtering**** (`fusion\_features.py:126-151`): Every call in the suite passes `court\_polygon=None` (test lines 86, 88, 89, and the line-117 aggregator call), so the off-court filter guard at line 141 is dead in tests. A `NEAR\_ONLY` polygon correctly collapses two_sided from `1.0` to `0.0`, but nothing asserts it.

****Why I downgraded med ? low****: This is a missing-assertion gap, not a shipped defect. The feature math is correct, the paths run at runtime via `compute\_person\_features`, the polygon primitives are independently tested with `LEFT\_HALF` (test lines 24-63), and the segmenter is default-OFF. The finding is precise (it correctly scopes the gap to the *integration through* `two\_sided\_motion`, not the primitive) and actionable, but nothing is broken ? so it's a legitimate hardening item rather than a medium-severity issue.

The proposed fix is correct and minimal.